

OnSale — архитектура: как это работает

Документ для разработчика и администратора. Руководства для пользователей по ролям — рядом: `admin.md`, `agent.md`, `analyst.md`, `buyer.md`, `courier.md`, `seller.md`. Остальная техническая документация:

Файл	О чём
<code>DATA-MODEL.md</code>	таблицы, поля, связи, инварианты данных
<code>API.md</code>	все серверные действия (server actions) и HTTP-маршруты: кто может вызвать, что проверяется, что возвращается
<code>OPERATIONS.md</code>	запуск, миграции БД, деплой, бэкап и откат, демо-стенд, тесты

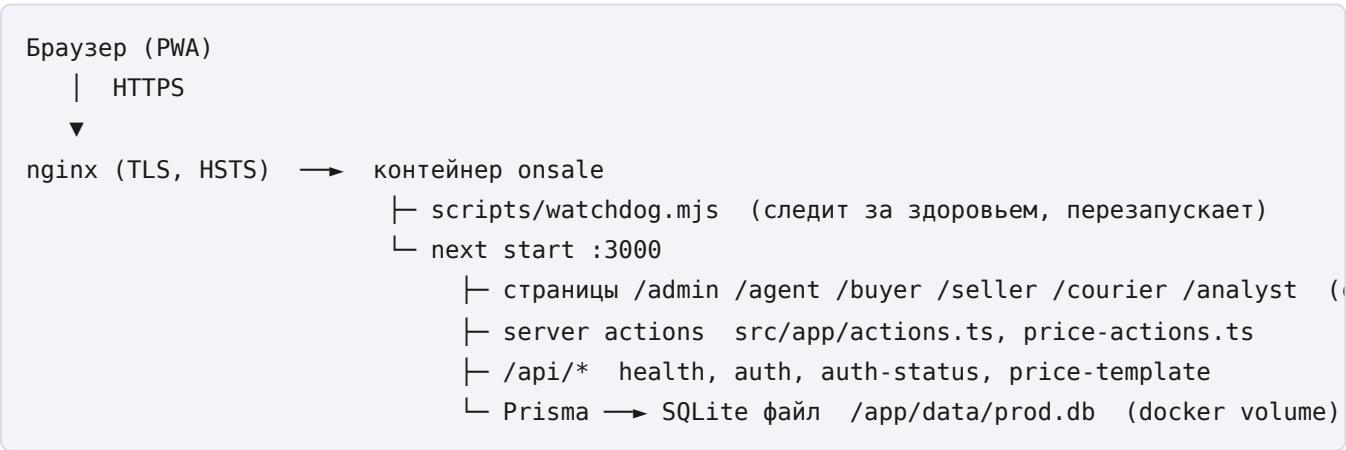
1. Что это

OnSale — веб-приложение оптовых продаж: покупатели и торговые агенты собирают заказы по прайс-листу, продавец ведёт заказ по статусам и принимает оплаты, курьер отмечает доставку, аналитик смотрит отчёты, администратор управляет пользователями и каталогом. Работает с телефона как PWA.

Стек: Next.js 16 (App Router, React 19, server actions), TypeScript, Tailwind 4, Prisma 6 + SQLite, NextAuth v4 (вход по email/паролю, JWT-сессии), zod 4 (валидация), vitest (тесты), recharts (графики), exceljs/raparaparse (импорт прайс-листов). Деплой — Docker, один контейнер, один файл БД.

> В `AGENTS.md` указано, что этот Next.js отличается от привычного: перед изменением кода, который использует API Next, читайте `node_modules/next/dist/docs/`.

2. Общая схема



Ключевые принципы:

- Всё чтение — в серверных компонентах** (`page.tsx`): они напрямую обращаются к Prisma и `src/lib/stats.ts`. Клиентские компоненты (`"use client"`) получают готовые данные пропсами.
- Вся запись — через server actions.** Клиент не обращается к БД; каждое действие само проверяет сессию, роль и права на конкретную запись.

3. **Сервер никому не доверяет.** Аргументы действий проверяются zod-схемами (`src/lib/validation.ts`), права — заново на каждый вызов, а не по состоянию интерфейса.
4. **Любая денежная/статусная операция — одна транзакция** с записью в журнал.

3. Структура репозитория

```
prisma/
  schema.prisma          модель данных
  migrations/             SQL-миграции (baseline + аддитивные)
  seed.ts                 создаёт администратора и справочник причин корректировки
src/
  app/
    layout.tsx, page.tsx  корень; "/" перенаправляет по роли (см. §4)
    login/                страница входа + LoginForm (обратный отсчёт блокировки)
    admin|agent|buyer|seller|courier|analyst/
                                по каталогу на роль: layout.tsx (requireRole), page.tsx и подстраницы
                                loading.tsx / error.tsx
    actions.ts            server actions: заказы, оплаты, пользователи, товары, категории
    price-actions.ts      server actions: импорт прайс-листа (2 фазы)
    api/                  health, auth/[...nextauth], auth-status, price-template
    manifest.ts           PWA-манифест
  components/            клиентские и общие компоненты (формы, таблицы, Toaster, ui.tsx ...)
  lib/                   бизнес-логика и инфраструктура (см. ниже)
  actions.integration.test.ts  интеграционные тесты actions на настоящей SQLite
public/                  иконки, sw.js (service worker), offline.html
scripts/
  db-migrate.mjs          безопасная миграция БД при старте
  watchdog.mjs            процесс-надсмотрщик внутри контейнера
  gen-icons.mjs, make-dist.mjs, md2pdf.mjs  служебные
docker-entrypoint.sh, Dockerfile, docker-compose.yml
```

Модули `src/lib`

Модуль	Назначение
<code>auth.ts</code>	настройки NextAuth: вход, лимит попыток, проверка блокировки, обновление роли в JWT
<code>rbac-core.ts</code>	роли, статусы заказа и их подписи, константы статусов, чистый расчёт долга/просрочки <code>computeOrderDebtInfo</code>
<code>rbac.ts</code>	<code>getSessionUser()</code> (null, если сессия отозвана), <code>requireRole(...)</code> для страниц (редирект)
<code>order-rules.ts</code>	машина статусов: <code>transitionError</code> , <code>statusOptions</code> , <code>statusAfterPayments</code> , <code>isOverpaid</code> — чистые функции без БД
<code>validation.ts</code>	zod-схемы всех входных данных и <code>parse(schema, data)</code> (бросает <code>UserError</code> с понятным текстом)
<code>action-result.ts</code>	<code>ActionResult<T></code> , <code>UserError</code> , <code>unwrap()</code>
<code>db-lock.ts</code>	очередь записи SQLite: <code>withWriteLock</code> , <code>writeTx</code>
<code>money.ts</code>	<code>roundMoney</code> , <code>moneyGte/Gt</code> , <code>MONEY_EPS = 0.005</code> , <code>fmtMoney</code>
<code>tz.ts</code> , <code>format.ts</code>	часовой пояс бизнеса <code>Asia/Novosibirsk</code> , форматирование дат и рублей
<code>rate-limit.ts</code>	ограничитель попыток входа в памяти процесса
<code>stats.ts</code>	все отчёты и агрегаты (долги, агенты, товары, рейтинг)
<code>price-parse.ts</code> , <code>article.ts</code>	разбор CSV/XLSX прайса; артикулы «APT-000001»
<code>password.ts</code> , <code>prisma.ts</code>	проверка пароля; singleton PrismaClient

4. Роли и доступ

Шесть ролей: `BUYER` (покупатель), `AGENT` (торговый агент), `SELLER` (продавец), `COURIER` (курьер), `ANALYST` (аналитик), `ADMIN` (администратор).

- Главная / перенаправляет на `/buyer`, `/agent`, `/seller`, `/courier`, `/analyst`, `/admin` по роли; без сессии — на `/login`.
- Доступ к страницам:** `layout.tsx` каждого каталога вызывает `requireRole("<Роль>")`. Чужая роль → редирект на `/`.
- Доступ к данным и действиям:** каждое server action вызывает `assertRole(...)`; дополнительно проверяется принадлежность записи (покупатель — только свои заказы, агент — только заказы своих покупателей). Скрытая кнопка в интерфейсе безопасностью не считается.

Роль	Что делает
BUYER	каталог, свои заказы (черновик → передача в работу, отмена до обработки), свои оплаты и долг, профиль (адрес, телефон)
AGENT	то же от имени закреплённых покупателей; статистика своих клиентов
SELLER	ведёт заказы по статусам (в т.ч. массово), принимает/корректирует/удаляет оплаты, правит состав заказа до отгрузки; маршрутный лист
COURIER	единственное действие: «Отгружен» → «Доставлен»
ANALYST	только чтение: отчёты по агентам, товарам, рейтинг, прайс-лист
ADMIN	всё: пользователи (создание, роль, пароль, блокировка, удаление), прайс-лист и товары, категории, справочник причин, пометка заказов на удаление, любые корректировки

5. Аутентификация и сессии

- NextAuth v4, провайдер Credentials (email + пароль, хэш bcrypt). Email хранится в нижнем регистре.
- Сессия — JWT в cookie, срок 7 дней. **Роль и состояние пользователя не берутся из токена «на веру»:** при каждом чтении сессии callback `jwt` перечитывает из БД `role`, `blocked`, `sessionVersion`. Если пользователь удалён, заблокирован или его `sessionVersion` не совпадает с токеном — токен помечается `revoked`, `getSessionUser()` возвращает `null`, действие отвечает «Сессия истекла — войдите заново». Смена роли действует сразу.
- `sessionVersion` увеличивается при смене пароля и при блокировке — все выданные ранее сессии пользователя умирают.
- **Защита входа** (`rate-limit.ts`, память процесса): 3 неудачи подряд на email и 20 на IP → блокировка с ростом 30 с → 1 мин → 2 мин ... до 1 часа. IP берётся из `x-real-ip`, иначе из первого `x-forwarded-for`. Если пользователя нет, всё равно выполняется сравнение с фиктивным хэшем — время ответа не выдаёт, существует ли email. Состояние лимита обнуляется при перезапуске контейнера.
- `POST /api/auth-status` отдаёт форме входа оставшиеся попытки/время блокировки (без секретов).

6. Серверные действия: общий каркас

Файл `src/app/actions.ts` (директива `"use server"`). Все действия устроены одинаково:

```
export async function someAction(args) {
  return run(async () => {
    const me = await assertRole("SELLER", "ADMIN"); // 1. сессия и роль
    const input = parse(someSchema, args); // 2. zod-валидация
    await writeTx(async (tx) => { // 3. транзакция в очереди записи
      ...проверки состояния + изменения + запись в журнал...
    });
    revalidateOrders(); // 4. сброс кэша страниц
    return { ...полезные данные };
  });
}
```

Контракт результата — `ActionResult<T>`: `{ ok: true, ...данные }` или `{ ok: false, error: "текст для пользователя" }`. Ожидаемые ошибки (валидация, права, бизнес-правила — это `UserError`) **возвращаются значением**, а не бросаются: в production Next.js прячет текст брошенных исключений за «An error occurred». Прочие ошибки логируются (`[action]`) и превращаются в «Внутренняя ошибка...». Особые случаи: `P2025` → «Запись не найдена — обновите страницу»; занятая БД (`P2034`, `SQLITE_BUSY`) → «Данные одновременно меняет другой пользователь. Повторите операцию.».

На клиенте: `unwrap(await action(...))` возвращает данные или бросает `Error` с текстом ошибки; интерфейс показывает его через `toast.error(...)` (`components/Toaster.tsx`).

Полный список действий — в `API.md`.

7. Параллельная запись и SQLite

SQLite допускает одного писателя. Интерактивные транзакции Prisma при конкуренции ждут друг друга до таймаута и падают («Socket timeout»). Поэтому `db-lock.ts` выстраивает все пишущие транзакции в **очередь внутри процесса** (`writeTx`, параметры по умолчанию: ожидание 10 с, выполнение 20 с). Следствия:

- приложение должно работать **одним процессом/контейнером** (так и настроено); масштабирование на несколько реплик без смены БД не поддерживается;
- двойной клик или два продавца одновременно — операции выполняются строго по очереди;
- номер заказа = `max(number)+1` внутри транзакции; при конфликте уникальности создание повторяется до 5 раз;
- импорт прайса идёт одной долгой транзакцией (таймаут 120 с).

8. Правила заказов и оплат

Реализованы в `order-rules.ts` (чистые функции; покрыты тестами) и применяются внутри транзакций `actions.ts`.

Статусы: `NEW` (черновик) → `ENTERED` (внесён) → `ASSEMBLED` (собран) → `SHIPPED` (отгружен) → `DELIVERED` (доставлен) → `PAID` (оплачен); отдельно `CANCELLED`.

Кто и как меняет статус (`transitionError`):

- **Покупатель/агент:** `NEW` → `ENTERED` (передать в работу, `submitOrder`) и `NEW|ENTERED` → `CANCELLED` (`cancelOrder`).

- **Курьер:** только `SHIPPED` → `DELIVERED` .
- **Продавец:** вперёд по цепочке `NEW...DELIVERED` — с пропуском шагов; назад — только на один шаг. В `NEW` вернуть нельзя.
- **Администратор:** то же, но назад — на любой шаг (кроме `NEW`).
- **PAID** ставится **автоматически**, когда сумма оплат ≥ сумма заказа (и заказ в статусе, допускающем оплату). Вручную в `PAID` не перевести, из `PAID` вручную не вывести: статус откатывается, только если оплату скорректировали или удалили — к статусу, который был до оплаты, а если он неизвестен — к `DELIVERED` .
- **CANCELLED** : только из `NEW` / `ENTERED` / `ASSEMBLED` и только если по заказу нет оплат. Восстановить (в `ENTERED`) может только `ADMIN` .
- Если по заказу есть оплаты, откатить его в статус, не допускающий оплат, нельзя.

Оплаты:

- принимаются только по заказам в статусах `ASSEMBLED` , `SHIPPED` , `DELIVERED` ; сумма не может превышать остаток долга по заказу;
- допускаются «оплаты без заказа» (аванс на покупателя) — они фиксируются в общем `AuditLog` ;
- **продавец** может корректировать и удалять только оплаты, внесённые «сегодня» (по часовому поясу бизнеса); более старые — только администратор;
- корректировка и удаление требуют текст причины (≥ 3 символов) и пишутся в журнал;
- после любой операции с оплатами или составом заказа вызывается `syncPaymentStatus` — перевод в/из `PAID` и проверка, что оплаты не превысили сумму.

Корректировка состава (`editOrderItems`): только до отгрузки (`NEW/ENTERED/ASSEMBLED`), только уменьшение/изменение количества (`qty=0` — удаление позиции), нельзя удалить все позиции (тогда — отмена), нельзя, чтобы новая сумма стала меньше уже оплаченного. Цены позиций фиксируются в момент создания заказа (снимок), изменение прайса старые заказы не затрагивает.

Долг и просрочка (`computeOrderDebtInfo`): долг = сумма – оплачено; срок оплаты = дата создания + отсрочка покупателя (`User.deferral` , дней); просрочен, если статус в `SHIPPED/DELIVERED/PAID` , остаток > 0 и срок прошёл. В отчёты продаж входят статусы `ENTERED...PAID` ; черновики и отменённые — нет; помеченные на удаление (`deleted`) скрыты везде.

9. Деньги и время

- **Деньги — Float в рублях.** Любое значение округляется `roundMoney` до копеек, сравнения идут с допуском `MONEY_EPS = 0.005` (`moneyGte/moneyGt`). Не сравнивайте суммы через `===` . (Перевод на целые копейки — возможное улучшение, пока сознательно не делался: потребует миграции данных.)
- **Часовой пояс бизнеса — Asia/Novosibirsk** (`tz.ts` , `APP_TIME_ZONE`). В БД даты хранятся в UTC; «сегодня», границы периодов отчётов (`periodRange` , `startOfBusinessDay` , `endOfBusinessDay`), правило «оплата день в день» и форматирование (`format.ts`) считаются в этом поясе независимо от пояса сервера и браузера.

10. Импорт прайс-листа

Двухфазный (`price-actions.ts` , только `ADMIN`):

1. `stagePriceList(FormData)` — файл CSV/XLSX до 15 МБ разбирается (`price-parse.ts`: определение кодировки UTF-8/Windows-1251, разделителя, сопоставление колонок `article/` артикул, `name/`наименование, `unit, price, stock, category, manufacturer`). Возвращает строки и список категорий; **в БД ничего не пишется**.
2. Интерфейс показывает категории и просит решить по новым: создать или влить в существующую. `applyPriceList({products, categoryChoices})` повторно валидирует каждую строку (данные пришли с клиента), затем **одной транзакцией**: создаёт/обновляет категории, строки без артикула получают «APT-N+1», строки с артикулом обновляются (`upsert` по артикулу). Любая ошибка — откат всего импорта. Лимит — 20 000 строк.

Шаблон файла: `GET /api/price-template?format=csv|xlsx` (только `ADMIN`).

11. Интерфейс

- Общая оболочка `AppShell` (меню по роли, активный пункт подсвечивается `NavLinks`), примитивы `ui.tsx` (`PageHeader`, `StatCard`, `Card`, `EmptyState`, `BackLink`), уведомления `Toaster` (`toast.success/error/info/fromError`).
- Палитра — токены `brand-50...900` в `globals.css`; шрифт — системный стек.
- В каждом каталоге роли есть `loading.tsx` (скелетон) и `error.tsx` (аккуратная ошибка с «Повторить»); общий `not-found.tsx`.
- Список допустимых статусов в `StatusSelect` строится из тех же `statusOptions`, что и проверка на сервере, поэтому пользователь видит только разрешённые переходы.
- PWA: `manifest.ts`, `public/sw.js`, `offline.html`, `InstallPrompt`.

12. Журналы (аудит)

Журнал	Что фиксирует
<code>OrderStatusLog</code>	каждая смена статуса: статус, кто, когда, примечание
<code>OrderAuditLog</code>	по заказу: <code>payment_added</code> , <code>payment_edited</code> , <code>payment_deleted</code> , <code>order_edited</code> + сумма и описание
<code>AuditLog</code>	общее: оплаты без заказа, <code>user_updated</code> , <code>user_blocked/unblocked</code> , <code>user_deleted</code>

Записи пишутся в той же транзакции, что и изменение, поэтому не расходятся с данными.

13. Заголовки безопасности

`next.config.ts`: `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy`, `Content-Security-Policy` (`default-src 'self'; 'unsafe-eval'` только в dev). HSTS выставляет `nginx` (приложение может работать и по HTTP в локальной сети). Лимит тела `server action` — 15 МБ (для прайсов).